

## WEEK-03(TASKS 1 to 5)

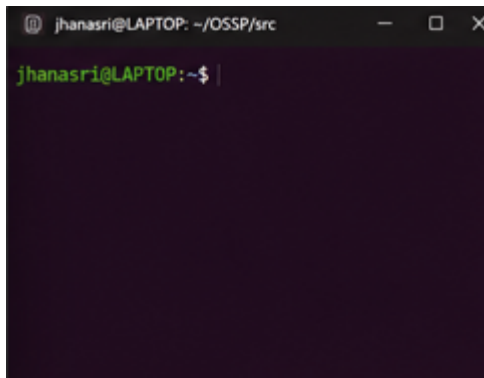
### Task 1 :

## Step 1: Open Ubuntu (WSL)

Click **Start** → Search **Ubuntu** → Open it.

You should see something like:

jhanasri@LAPTOP-XXXX:~\$



## Step 2: Go to Your Project Folder

Type:

```
cd ~/OSSP/src
```

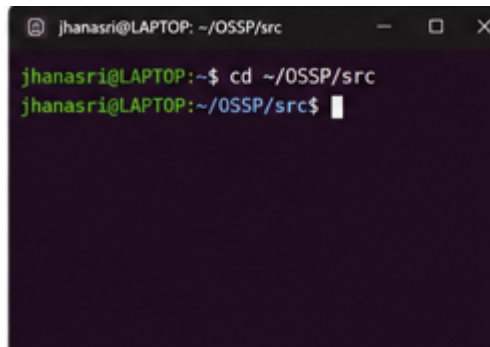
Press **Enter**.

Verify your location:

```
pwd
```

Expected output:

```
/home/jhanasri/OSSP/src
```

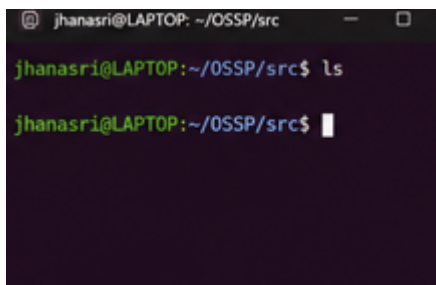
A terminal window with a dark background. The title bar shows 'jhanasri@LAPTOP: ~/OSSP/src'. The prompt is 'jhanasri@LAPTOP:~\$'. The user has entered 'cd ~/OSSP/src' and the prompt is now 'jhanasri@LAPTOP:~/OSSP/src\$' with a cursor.

```
jhanasri@LAPTOP: ~/OSSP/src
jhanasri@LAPTOP:~$ cd ~/OSSP/src
jhanasri@LAPTOP:~/OSSP/src$
```

## Step 3: Check Existing Files

ls

You'll see your existing files (if any).

A terminal window with a dark background. The title bar shows 'jhanasri@LAPTOP: ~/OSSP/src'. The prompt is 'jhanasri@LAPTOP:~/OSSP/src\$'. The user has entered 'ls' and the prompt is now 'jhanasri@LAPTOP:~/OSSP/src\$' with a cursor.

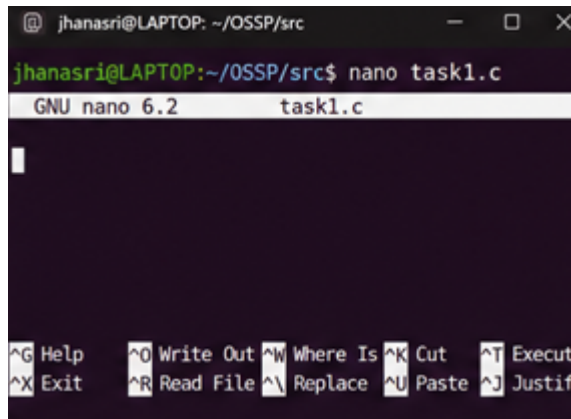
```
jhanasri@LAPTOP: ~/OSSP/src
jhanasri@LAPTOP:~/OSSP/src$ ls
jhanasri@LAPTOP:~/OSSP/src$
```

## Step 4: Create a New C File

nano task1.c

Press **Enter**.

Nano editor will open.



## Step 5: Paste the Code

Copy and paste this code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    char command[100];

    while (1)
    {
        printf("myshell> ");
        fgets(command, sizeof(command), stdin);

        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0)
        {
            printf("Exiting Shell...\n");
            break;
        }

        pid_t pid = fork();

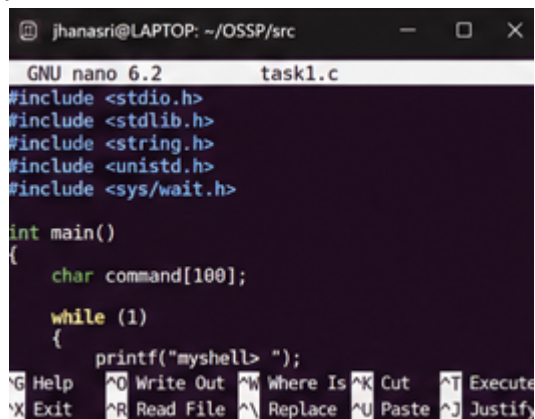
        if (pid == 0)
        {
            execlp(command, command, NULL);
```

```

        printf("Command not found!\n");
        exit(1);
    }
    else
    {
        wait(NULL);
    }
}

return 0;
}

```



```

jhanasri@LAPTOP: ~/OSSP/src
GNU nano 6.2      task1.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    char command[100];

    while (1)
    {
        printf("myshell> ");

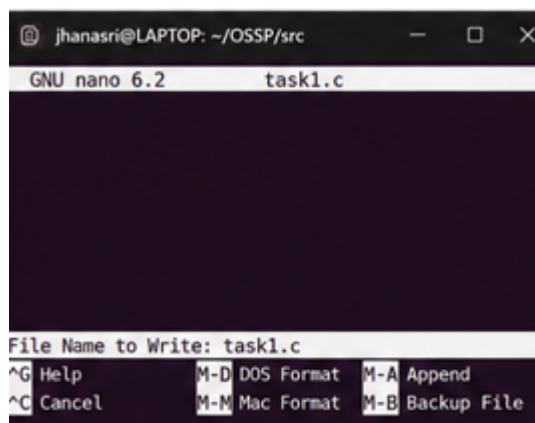
```

## Step 6: Save the File

Press:

Ctrl + O

Press **Enter**.



```

jhanasri@LAPTOP: ~/OSSP/src
GNU nano 6.2      task1.c

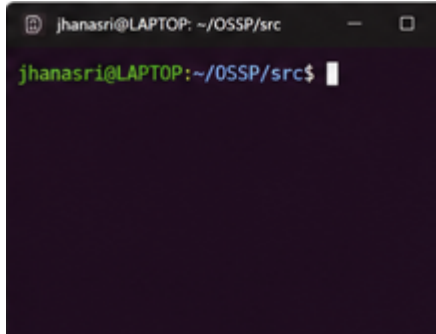
File Name to Write: task1.c
^G Help      ^M-D DOS Format  ^M-A Append
^C Cancel    ^M-M Mac Format  ^M-B Backup File

```

## Step 7: Exit Nano

Press:

Ctrl + X

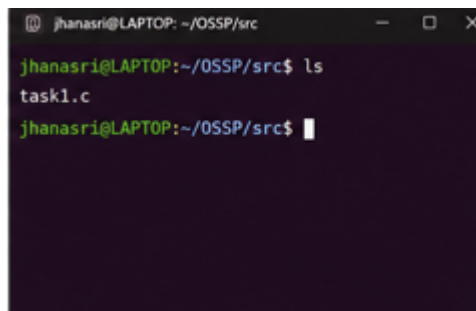
A terminal window with a dark background. The title bar shows 'jhanasri@LAPTOP: ~/OSSP/src'. The prompt is 'jhanasri@LAPTOP:~/OSSP/src\$' followed by a cursor.

## Step 8: Check the File Was Created

ls

Expected output:

task1.c

A terminal window with a dark background. The title bar shows 'jhanasri@LAPTOP: ~/OSSP/src'. The prompt is 'jhanasri@LAPTOP:~/OSSP/src\$'. The command 'ls' has been entered, and the output 'task1.c' is displayed on the next line. The prompt is now 'jhanasri@LAPTOP:~/OSSP/src\$' followed by a cursor.

## Step 9: Compile the Program

Type:

gcc task1.c -o task1

Press **Enter**.

If there are no errors, you'll return to the prompt.

```
jhanasri@LAPTOP: ~/OSSP/src
jhanasri@LAPTOP:~/OSSP/src$ gcc task1.c -o task1
jhanasri@LAPTOP:~/OSSP/src$
```

## Step 10: Verify the Executable

ls

Expected output:

task1

Task1.c

```
jhanasri@LAPTOP: ~/OSSP/src
jhanasri@LAPTOP:~/OSSP/src$ ls
task1  task1.c
jhanasri@LAPTOP:~/OSSP/src$
```

## Step 11: Run the Program

./task1

You should see:

myshell>

```
jhanasri@LAPTOP: ~/OSSP/src
jhanasri@LAPTOP:~/OSSP/src$ ./task1
myshell>
```

# Step 12: Test Commands

Try these one by one.

## Check Current Directory

```
myshell> pwd
```

Example:

```
/home/jhanasri/OSSP/src
```

## List Files

```
myshell> ls
```

Example:

```
task1  
task1.c
```

## Show Current Date

```
myshell> date
```

Example:

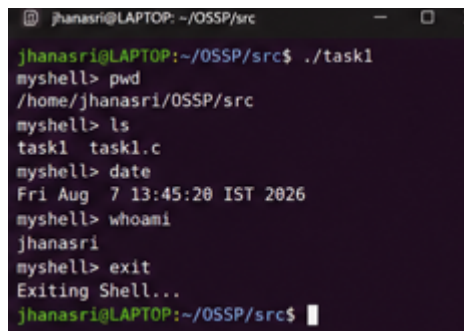
```
Fri Aug 7 13:45:20 IST 2026
```

## Display Username

```
myshell> whoami
```

Example:

```
jhanasri
```



```
jhanasri@LAPTOP: ~/OSSP/src  
jhanasri@LAPTOP:~/OSSP/src$ ./task1  
myshell> pwd  
/home/jhanasri/OSSP/src  
myshell> ls  
task1 task1.c  
myshell> date  
Fri Aug 7 13:45:20 IST 2026  
myshell> whoami  
jhanasri  
myshell> exit  
Exiting Shell...  
jhanasri@LAPTOP:~/OSSP/src$
```

## Step 13: Exit the Shell

Type:

```
myshell> exit
```

Output:

Exiting Shell...

The program terminates successfully.

---

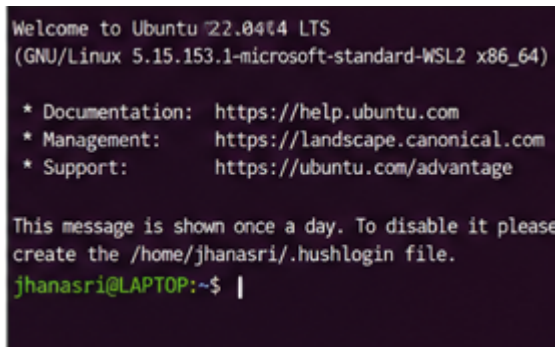
## Task 2:

## Step 1: Open Ubuntu

Open **Ubuntu (WSL)**.

You should see:

```
jhanasri@LAPTOP-XXXX:~$
```



```
Welcome to Ubuntu 22.04 LTS
(GNU/Linux 5.15.153.1-microsoft-standard-WSL2 x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

This message is shown once a day. To disable it please
create the /home/jhanasri/.hushlogin file.
jhanasri@LAPTOP:~$
```

## Step 2: Go to Your Project Folder

```
cd ~/OSSP/src
```

Check:



pwd

Expected:

/home/jhanasri/OSSP/src

```
jhanasri@LAPTOP:~$ cd ~/OSSP/src
jhanasri@LAPTOP:~/OSSP/src$ pwd
/home/jhanasri/OSSP/src
jhanasri@LAPTOP:~/OSSP/src$
```

## Step 3: Create File1.txt

Type:

nano File1.txt

Paste:

Hello Everyone  
Welcome to OSSP Lab  
This is Task 2  
Using Linux System Calls

Save:

- **Ctrl + O**
- **Enter**
- **Ctrl + X**

```
GNU nano 6.2  File1.txt
Hello Everyone
Welcome to OSSP Lab
This is Task 2
Using Linux System Calls

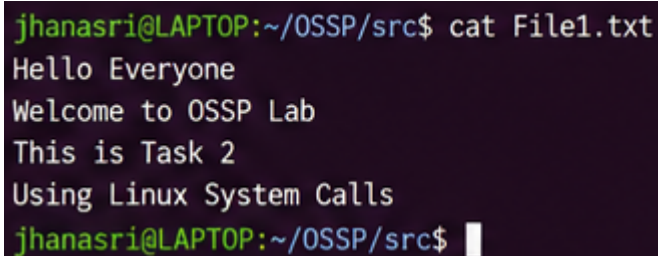
^G Help  ^O Write Out ^W Where Is ^K Cut  ^T Execute
^X Exit  ^R Read File ^\ Replace ^U Paste ^J Justify
```

## Step 4: Verify File1.txt

cat File1.txt

Output:

Hello Everyone  
Welcome to OSSP Lab  
This is Task 2  
Using Linux System Calls

A terminal window with a dark purple background. The prompt is 'jhanasri@LAPTOP:~/OSSP/src\$'. The command 'cat File1.txt' has been entered and executed. The output is displayed on the following lines: 'Hello Everyone', 'Welcome to OSSP Lab', 'This is Task 2', and 'Using Linux System Calls'. The prompt is now 'jhanasri@LAPTOP:~/OSSP/src\$' with a cursor at the end.

```
jhanasri@LAPTOP:~/OSSP/src$ cat File1.txt
Hello Everyone
Welcome to OSSP Lab
This is Task 2
Using Linux System Calls
jhanasri@LAPTOP:~/OSSP/src$
```

## Step 5: Create the C Program

nano task2.c

Paste the following code:

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>

int main()
{
    int source, destination;
    char buffer[100];
    int bytesRead;

    source = open("File1.txt", O_RDONLY);

    if(source < 0)
    {
        printf("Unable to open File1.txt\n");
        return 1;
    }
}
```

```

destination = open("File2.txt", O_CREAT | O_WRONLY | O_TRUNC, 0644);

if(destination < 0)
{
    printf("Unable to create File2.txt\n");
    close(source);
    return 1;
}

while((bytesRead = read(source, buffer, sizeof(buffer))) > 0)
{
    write(destination, buffer, bytesRead);
}

close(source);
close(destination);

printf("File copied successfully.\n");

return 0;
}

```

```

GNU nano 6.2      task2.c
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>

int main()
{
    int source, destination;
    char buffer[100];
    int bytesRead;
}

```

^G Help   ^O Write Out   ^W Where Is   ^K Cut   ^T Execute  
 ^X Exit   ^R Read File   ^\ Replace   ^U Paste   ^J Justify

## Step 6: Save the Program

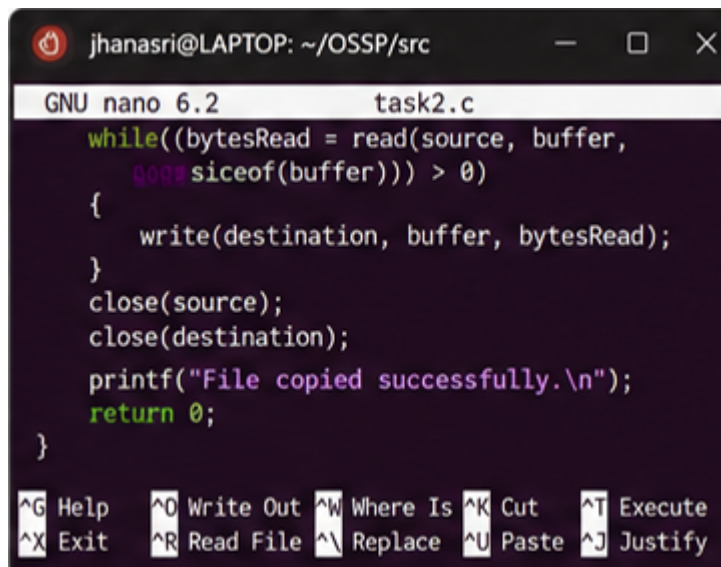
Press:

Ctrl + O

Press **Enter**

Then:

Ctrl + X



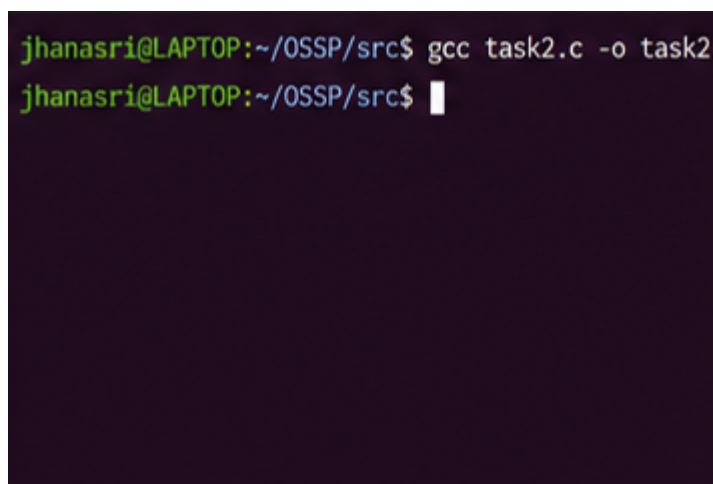
```
jhanasri@LAPTOP: ~/OSSP/src
GNU nano 6.2 task2.c
while((bytesRead = read(source, buffer,
sizeof(buffer))) > 0)
{
    write(destination, buffer, bytesRead);
}
close(source);
close(destination);
printf("File copied successfully.\n");
return 0;
}

^G Help      ^O Write Out ^W Where Is  ^K Cut      ^T Execute
^X Exit      ^R Read File ^\ Replace   ^U Paste    ^J Justify
```

## Step 7: Compile

`gcc task2.c -o task2`

If no errors occur, compilation is successful.



```
jhanasri@LAPTOP:~/OSSP/src$ gcc task2.c -o task2
jhanasri@LAPTOP:~/OSSP/src$
```

## Step 8: Run the Program

`./task2`

Output:

File copied successfully.

```
jhanasri@LAPTOP:~/OSSP/src$ ./task2
File copied successfully.
jhanasri@LAPTOP:~/OSSP/src$
```

## Step 9: Verify File2.txt

cat File2.txt

Output:

Hello Everyone  
Welcome to OSSP Lab  
This is Task 2  
Using Linux System Calls

```
jhanasri@LAPTOP:~/OSSP/src$ cat File2.txt
Hello Everyone
Welcome to OSSP Lab
This is Task 2
Using Linux System Calls
jhanasri@LAPTOP:~/OSSP/src$
```

## Step 10: Verify Both Files

ls

Expected:

File1.txt  
File2.txt

task2  
task2.c

```
jhanasri@LAPTOP:~/OSSP/src$ ls -l
total 28
-rw-rw-r-- 1 jhanasri jhanasri 76 May  8 11:12 File1.txt
-rw-rw-r-- 1 jhanasri jhanasri 76 May  8 11:12 File2.txt
-rwxrwxr-x 1 jhanasri jhanasri 16640 May  8 11:12 task2
-rw-rw-r-- 1 jhanasri jhanasri 1004 May  8 11:10 task2.c
jhanasri@LAPTOP:~/OSSP/src$
```

## Step 11: Explanation of System Calls

| System Call          | Purpose                                               |
|----------------------|-------------------------------------------------------|
| <code>open()</code>  | Opens an existing file or creates a new file.         |
| <code>read()</code>  | Reads data from the source file into a buffer.        |
| <code>write()</code> | Writes the buffer contents into the destination file. |
| <code>close()</code> | Closes the opened files and releases resources.       |

These are the same system calls and usage described in your lab document.

```
jhanasri@LAPTOP:~/OSSP/src$ wc -c File1.txt File2.txt
 76 File1.txt
 76 File2.txt
152 total
jhanasri@LAPTOP:~/OSSP/src$
```

```
jhanasri@LAPTOP:~/OSSP/src$ cat File1.txt; echo;  
cat File2.txt  
Hello Everyone  
Welcome to OSSP Lab  
This is Task 2  
Using Linux System Calls  
  
Hello Everyone  
Welcome to OSSP Lab  
This is Task 2  
Using Linux System Calls  
jhanasri@LAPTOP:~/OSSP/src$
```

---

### Task 3:

## Step 1: Open Ubuntu (WSL)

Open **Ubuntu** from the Windows Start Menu.

Expected screen:

jhanasri@LAPTOP-XXXX:~\$

```
Welcome to Ubuntu 22.04.4 LTS  
(GNU/Linux 5.15.153.1-microsoft-standard-WSL2 x86_64)  
  
* Documentation:  https://help.ubuntu.com  
* Management:    https://landscape.canonical.com  
* Support:        https://ubuntu.com/advantage  
  
This message is shown once a day. To disable it please  
create the /home/jhanasri/.hushlogin file.  
jhanasri@LAPTOP:~$
```

## Step 2: Go to the Project Folder

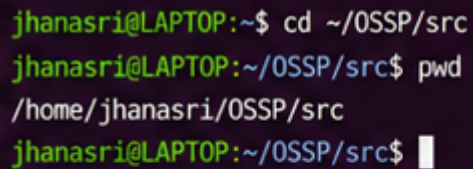
```
cd ~/OSSP/src
```

Verify:

```
pwd
```

Expected output:

```
/home/jhanasri/OSSP/src
```

A terminal window with a dark purple background. The prompt is 'jhanasri@LAPTOP:~\$'. The first command is 'cd ~/OSSP/src' and the second is 'pwd'. The output of 'pwd' is '/home/jhanasri/OSSP/src'.

```
jhanasri@LAPTOP:~$ cd ~/OSSP/src
jhanasri@LAPTOP:~/OSSP/src$ pwd
/home/jhanasri/OSSP/src
jhanasri@LAPTOP:~/OSSP/src$
```

## Step 3: Create a New C Program

```
nano task3.c
```



```
jhanasri@LAPTOP:~/OSSP/src$ nano task3.c
```

## Step 4: Paste the Code

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
#include <termios.h>
```

```
#include <string.h>
```

```
int main()
```

```
{
```

```
    struct termios oldt, newt;
```

```
    char ch;
```

```
    char buffer[100];
```

```
    int index = 0;
```

```
    // Save current terminal settings
```

```
    tcgetattr(STDIN_FILENO, &oldt);
```

```
    newt = oldt;
```

```
// Disable canonical mode and echo
newt.c_lflag &= ~(ICANON | ECHO);
tcsetattr(STDIN_FILENO, TCSANOW, &newt);

write(STDOUT_FILENO, "myshell> ", 9);

while (1)
{
    read(STDIN_FILENO, &ch, 1);

    if (ch == '\n')
    {
        buffer[index] = '\0';
        write(STDOUT_FILENO, "\nYou entered: ", 14);
        write(STDOUT_FILENO, buffer, strlen(buffer));
        write(STDOUT_FILENO, "\n", 1);

        if (strcmp(buffer, "exit") == 0)
            break;

        index = 0;
        write(STDOUT_FILENO, "myshell> ", 9);
    }
    else if ((ch == 127 || ch == 8) && index > 0)
    {
        index--;
    }
}
```

```

        write(STDOUT_FILENO, "\b \b", 3);

    }

    else if (index < 99)

    {

        buffer[index++] = ch;

        write(STDOUT_FILENO, &ch, 1);

    }

}

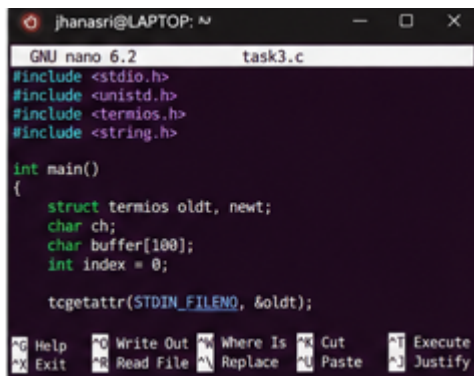
// Restore terminal settings

tcsetattr(STDIN_FILENO, TCSANOW, &oldt);

return 0;

}

```



```

jhanasri@LAPTOP: ~
GNU nano 6.2      task3.c
#include <stdio.h>
#include <unistd.h>
#include <termios.h>
#include <string.h>

int main()
{
    struct termios oldt, newt;
    char ch;
    char buffer[100];
    int index = 0;

    tcgetattr(STDIN_FILENO, &oldt);

```

## Step 5: Save the File

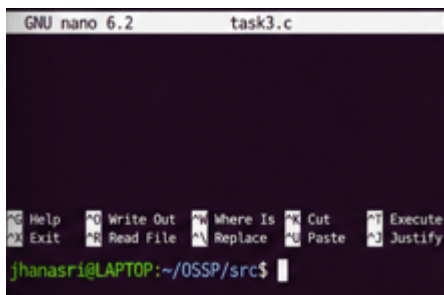
Press:

Ctrl + O

Press **Enter**

Then press:

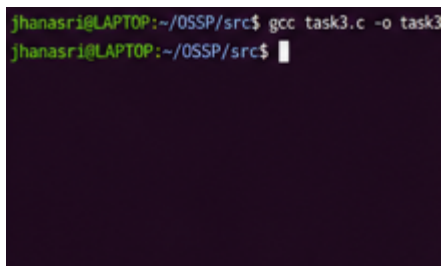
Ctrl + X



## Step 6: Compile the Program

```
gcc task3.c -o task3
```

If there are no errors, the program compiles successfully.

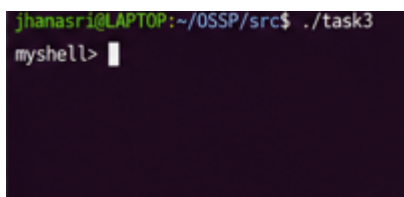


## Step 7: Run the Program

```
./task3
```

You should see:

```
myshell>
```



## Step 8: Test Keyboard Input

Type:

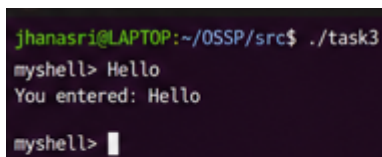
Hello

Press **Enter**

Output:

myshell> Hello

You entered: Hello

A terminal window with a dark purple background. The prompt is 'jhanasri@LAPTOP:~/OSSP/src\$ ./task3'. The user enters 'myshell> Hello'. The output is 'You entered: Hello'. The prompt returns to 'myshell> ' with a white cursor.

```
jhanasri@LAPTOP:~/OSSP/src$ ./task3
myshell> Hello
You entered: Hello
myshell> 
```

## Step 9: Test Backspace

Type:

Heloo

Press **Backspace** once, then type:

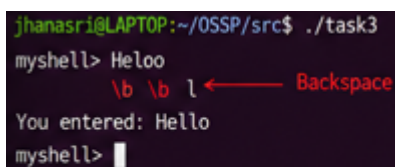
l

Press **Enter**

Output:

myshell> Hello

You entered: Hello

A terminal window with a dark purple background. The prompt is 'jhanasri@LAPTOP:~/OSSP/src\$ ./task3'. The user enters 'myshell> Heloo'. The next line shows '\b \b l' with a red arrow pointing to the second '\b' and the label 'Backspace'. The output is 'You entered: Hello'. The prompt returns to 'myshell> ' with a white cursor.

```
jhanasri@LAPTOP:~/OSSP/src$ ./task3
myshell> Heloo
\b \b l ← Backspace
You entered: Hello
myshell> 
```

## Step 10: Test Multi-Character Input

Type:

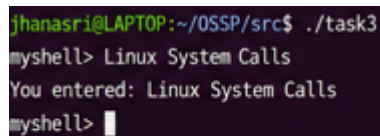
Linux System Calls

Press **Enter**

Output:

myshell> Linux System Calls

You entered: Linux System Calls



```
jhanasri@LAPTOP:~/OSSP/src$ ./task3
myshell> Linux System Calls
You entered: Linux System Calls
myshell> 
```

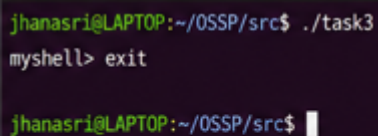
## Step 11: Exit the Program

Type:

exit

Press **Enter**

The program exits and returns to the Ubuntu prompt.



```
jhanasri@LAPTOP:~/OSSP/src$ ./task3
myshell> exit

jhanasri@LAPTOP:~/OSSP/src$ 
```

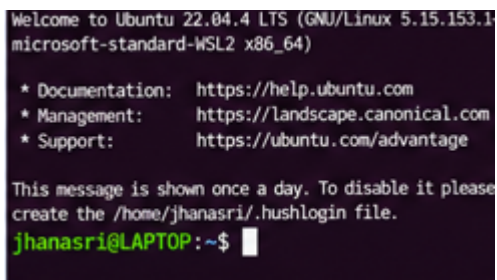
## **Task 4:**

### **Step 1: Open Ubuntu**

Open **Ubuntu** from the Windows Start Menu.

You should see:

jhanasri@LAPTOP-XXXX:~\$



```
Welcome to Ubuntu 22.04.4 LTS (GNU/Linux 5.15.153.1
microsoft-standard-WSL2 x86_64)

* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:        https://ubuntu.com/advantage

This message is shown once a day. To disable it please
create the /home/jhanasri/.hushlogin file.
jhanasri@LAPTOP:~$
```

### **Step 2: Go to your OSSP folder**

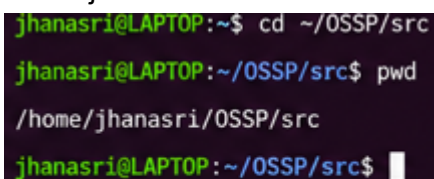
cd ~/OSSP/src

Check:

pwd

Expected:

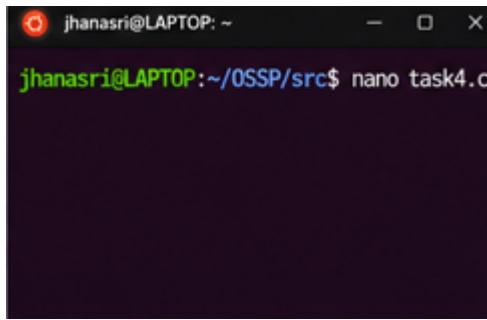
/home/jhanasri/OSSP/src



```
jhanasri@LAPTOP:~$ cd ~/OSSP/src
jhanasri@LAPTOP:~/OSSP/src$ pwd
/home/jhanasri/OSSP/src
jhanasri@LAPTOP:~/OSSP/src$
```

### **Step 3: Create Task 4 file**

nano task4.c



## Step 4: Paste this code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <termios.h>
#include <sys/wait.h>

#define MAX 100
#define HISTORY_SIZE 10

char history[HISTORY_SIZE][MAX];
int history_count = 0;
int history_index = 0;

void add_history(char *command)
{
    if (strlen(command) == 0)
        return;

    if (history_count < HISTORY_SIZE)
    {
        strcpy(history[history_count], command);
        history_count++;
    }
    else
    {
        for (int i = 1; i < HISTORY_SIZE; i++)
            strcpy(history[i - 1], history[i]);

        strcpy(history[HISTORY_SIZE - 1], command);
    }

    history_index = history_count;
}
```



```

void clear_line(int length)
{
    for (int i = 0; i < length; i++)
        write(STDOUT_FILENO, "\b \b", 3);
}

void execute_command(char *command)
{
    char *args[MAX];
    int count = 0;

    char *token = strtok(command, " ");

    while (token != NULL && count < MAX - 1)
    {
        args[count++] = token;
        token = strtok(NULL, " ");
    }

    args[count] = NULL;

    if (count == 0)
        return;

    if (strcmp(args[0], "exit") == 0)
        exit(0);

    pid_t pid = fork();

    if (pid == 0)
    {
        execvp(args[0], args);
        perror("Command not found");
        exit(1);
    }
    else
    {
        wait(NULL);
    }
}

int main()
{
    struct termios oldt, newt;

    char buffer[MAX];
    int index;

```

```

tcgetattr(STDIN_FILENO, &oldt);

newt = oldt;

newt.c_lflag &= ~(ICANON | ECHO);

tcsetattr(STDIN_FILENO, TCSANOW, &newt);

while (1)
{
    memset(buffer, 0, sizeof(buffer));
    index = 0;
    history_index = history_count;

    write(STDOUT_FILENO, "myshell> ", 9);

    while (1)
    {
        char ch;

        read(STDIN_FILENO, &ch, 1);

        /* Enter */
        if (ch == '\n')
        {
            buffer[index] = '\0';

            write(STDOUT_FILENO, "\n", 1);

            if (strlen(buffer) > 0)
                add_history(buffer);

            execute_command(buffer);

            break;
        }

        /* Backspace */
        else if ((ch == 127 || ch == 8) && index > 0)
        {
            index--;
            buffer[index] = '\0';

            write(STDOUT_FILENO, "\b \b", 3);
        }

        /* Escape sequence */
        else if (ch == 27)

```

```

{
    char seq[2];

    read(STDIN_FILENO, &seq[0], 1);
    read(STDIN_FILENO, &seq[1], 1);

    /* Up Arrow */
    if (seq[0] == '[' && seq[1] == 'A')
    {
        if (history_index > 0)
        {
            clear_line(index);

            history_index--;

            strcpy(buffer, history[history_index]);

            index = strlen(buffer);

            write(STDOUT_FILENO,
                buffer,
                index);
        }
    }

    /* Down Arrow */
    else if (seq[0] == '[' && seq[1] == 'B')
    {
        clear_line(index);

        if (history_index < history_count - 1)
        {
            history_index++;

            strcpy(buffer,
                history[history_index]);

            index = strlen(buffer);

            write(STDOUT_FILENO,
                buffer,
                index);
        }
        else
        {
            history_index = history_count;
            index = 0;
            buffer[0] = '\0';

```

```

    }
}

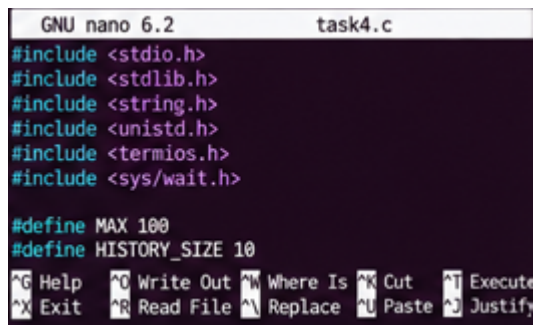
/* Normal character */
else if (index < MAX - 1)
{
    buffer[index++] = ch;

    write(STDOUT_FILENO, &ch, 1);
}
}
}

tcsetattr(STDIN_FILENO, TCSANOW, &oldt);

return 0;
}

```



```

GNU nano 6.2      task4.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <termios.h>
#include <sys/wait.h>

#define MAX 100
#define HISTORY_SIZE 10

^G Help  ^O Write Out  ^W Where Is  ^X Cut       ^T Execute
^X Exit  ^R Read File  ^_ Replace   ^U Paste     ^J Justify

```

## Step 5: Save the program

Press:

Ctrl + O

Press **Enter**.

Then:

Ctrl + X

```
jhanasri@LAPTOP:~/OSSP/src$
```

## Step 6: Compile

Run:

```
gcc task4.c -o task4
```

If there is no error, compilation is successful.

```
jhanasri@LAPTOP:~/OSSP/src$ gcc task4.c -o task4
jhanasri@LAPTOP:~/OSSP/src$
```

## Step 7: Run the program

```
./task4
```

You should see:

```
myshell>
```

```
jhanasri@LAPTOP:~/OSSP/src$ ./task4
myshell>
```

## Step 8: Test your first command

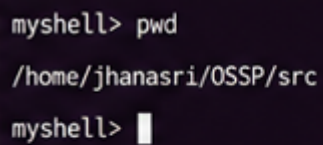
Type:

pwd

Press **Enter**.

Example:

```
myshell> pwd
/home/jhanasri/OSSP/src
myshell>
```

A screenshot of a terminal window with a dark background. It shows the command 'myshell> pwd' being entered, followed by the output '/home/jhanasri/OSSP/src' on the next line, and then 'myshell>' with a white cursor on the following line.

```
myshell> pwd
/home/jhanasri/OSSP/src
myshell> |
```

## Step 9: Enter another command

Type:

ls

Press **Enter**.

```
myshell> ls
task1
task1.c
task2
task2.c
task3
task3.c
task4
task4.c
myshell>
```

```
myshell> ls
task1    task1.c  task2    task2.c
task3    task3.c  task4    task4.c
myshell> █
```

## Step 10: Test command history

Now type:

date

Press **Enter**.

Then type:

whoami

Press **Enter**.

You now have commands stored in history:

pwd

ls

date

Whoami

```
myshell> date
Fri Aug  8 13:42:11 IST 2026
myshell> whoami
jhanasri
myshell> echo Hello World
Hello World
myshell> █
```

## Step 11: Test UP Arrow

At:

myshell>

Press the ↑ **Up Arrow** key.

The previous command should appear:

myshell> whoami

Press ↑ again:

myshell> date

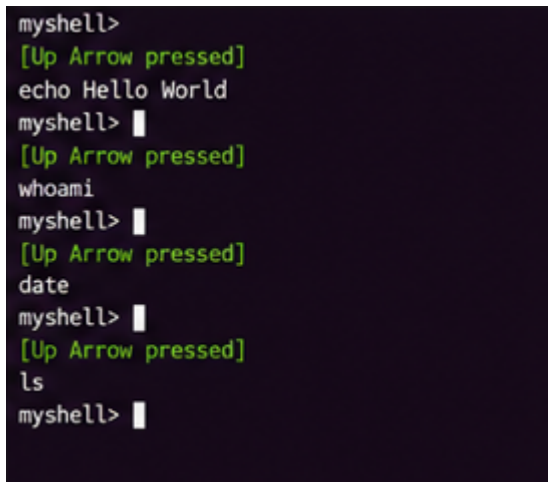
Press ↑ again:

myshell> ls

Press ↑ again:

myshell> pwd

This is the **previous-command navigation** required by Task 4.



```
myshell>
[Up Arrow pressed]
echo Hello World
myshell> 
[Up Arrow pressed]
whoami
myshell> 
[Up Arrow pressed]
date
myshell> 
[Up Arrow pressed]
ls
myshell>
```

## Step 12: Test DOWN Arrow

After using the Up Arrow, press:

↓

The next command in history should appear.

For example:

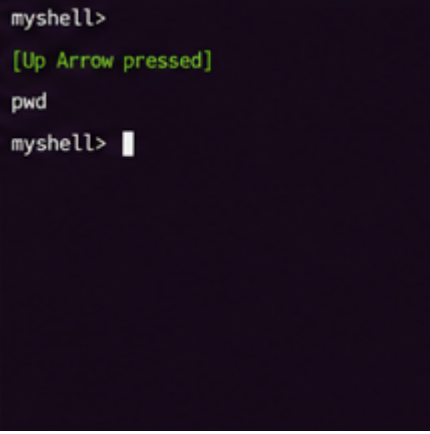


myshell> ls

Press ↓ again and the next command appears.

When you reach the newest position, the input becomes empty:

myshell>

A terminal window with a dark background. It shows the command history being navigated with the up arrow key. The first line is 'myshell>'. The second line is '[Up Arrow pressed]'. The third line is 'pwd'. The fourth line is 'myshell>' followed by a white cursor block.

```
myshell>
[Up Arrow pressed]
pwd
myshell> █
```

## Step 13: Test Backspace

Type:

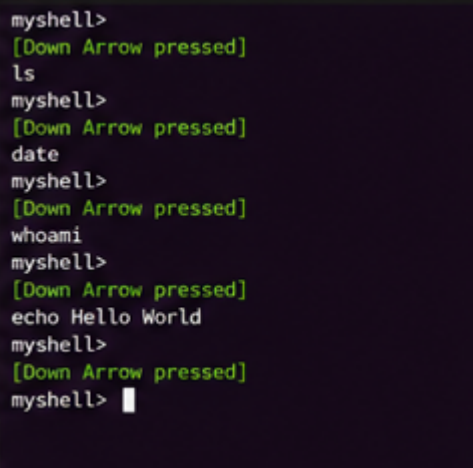
hellooo

Then press **Backspace** three times.

It should become:

hello

Press **Enter**.

A terminal window with a dark background showing a sequence of commands and history navigation. The first line is 'myshell>'. The second line is '[Down Arrow pressed]'. The third line is 'ls'. The fourth line is 'myshell>'. The fifth line is '[Down Arrow pressed]'. The sixth line is 'date'. The seventh line is 'myshell>'. The eighth line is '[Down Arrow pressed]'. The ninth line is 'whoami'. The tenth line is 'myshell>'. The eleventh line is '[Down Arrow pressed]'. The twelfth line is 'echo Hello World'. The thirteenth line is 'myshell>'. The fourteenth line is '[Down Arrow pressed]'. The fifteenth line is 'myshell>' followed by a white cursor block.

```
myshell>
[Down Arrow pressed]
ls
myshell>
[Down Arrow pressed]
date
myshell>
[Down Arrow pressed]
whoami
myshell>
[Down Arrow pressed]
echo Hello World
myshell>
[Down Arrow pressed]
myshell> █
```

## Step 14: Test a multi-word command

Type:

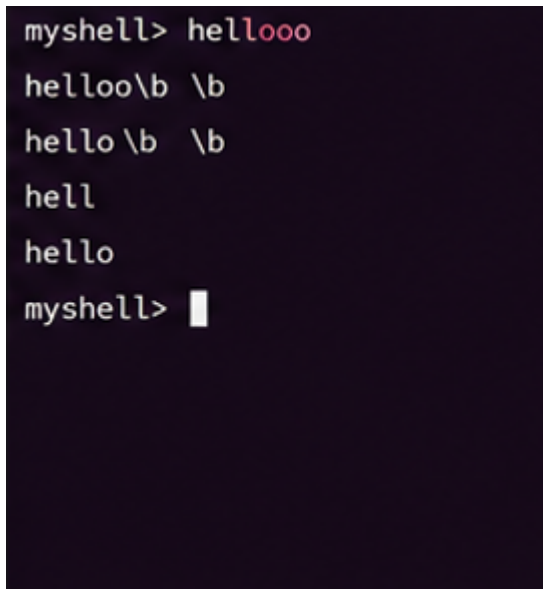
```
echo Hello World
```

Press **Enter**.

Output:

```
myshell> echo Hello World
Hello World
myshell>
```

The program tokenizes the command and passes the arguments to `execvp()`, matching the execution flow shown in the Task-4 diagram.



```
myshell> hellooo
hello\b \b
hello\b \b
hell
hello
myshell> 
```

## Step 15: Test exit

Type:

```
exit
```

Press **Enter**.

The program terminates and returns to:

```
jhanasri@LAPTOP-XXXX:~/OSSP/src$
```

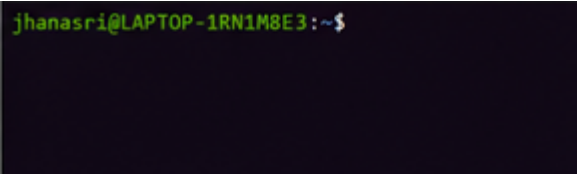
## **Task 5:**

### **Step 1: Open Ubuntu**

Open **Ubuntu** from the Windows Start Menu.

You should get:

jhanasri@LAPTOP-XXXX:~\$



```
jhanasri@LAPTOP-1RN1M8E3:~$
```

### **Step 2: Go to your OSSP folder**

Type:

```
cd ~/OSSP/src
```

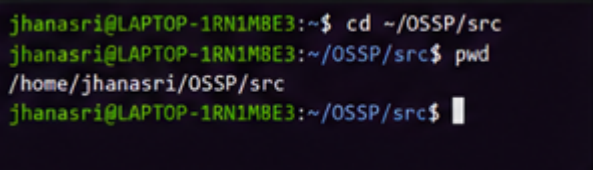
Press **Enter**.

Then check your location:

```
pwd
```

You should get:

```
/home/jhanasri/OSSP/src
```



```
jhanasri@LAPTOP-1RN1M8E3:~$ cd ~/OSSP/src
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/src$ pwd
/home/jhanasri/OSSP/src
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/src$
```

### **Step 3: Check your previous tasks**

Type:

```
ls
```

You should see files such as:

task1  
task1.c  
task2  
task2.c  
task3  
task3.c  
task4  
task4.c

```
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/src$ ls
task1  task1.c  task2  task2.c  task3  task3.c
task4  task4.c
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/src$
```

## Step 4: Create Task 5 C file

Type:

nano task5.c

Press **Enter**.

Nano will open.

```
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/src$ nano task5.c
|
```

## Step 5: Enter the Task 5 program

Paste this code:

```
#define _GNU_SOURCE
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <unistd.h>
```

```
#include <sys/mman.h>
```

```
struct Node
```

```
{
```

```

int data;
struct Node *next;
};

int main()
{
    /* -----
       PART 1: Dynamic Buffer
       ----- */

    size_t size = 100;

    char *buffer = mmap(
        NULL,
        size,
        PROT_READ | PROT_WRITE,
        MAP_PRIVATE | MAP_ANONYMOUS,
        -1,
        0
    );

    if (buffer == MAP_FAILED)
    {
        perror("mmap");
        return 1;
    }

    printf("Allocated 100 bytes using mmap()\n");

    printf("Enter some text: ");

    ssize_t n = read(STDIN_FILENO, buffer, size - 1);

    if (n < 0)
    {
        perror("read");
        munmap(buffer, size);
        return 1;
    }

    buffer[n] = '\0';

    printf("Input: %s", buffer);

    /* -----
       PART 2: Resize Buffer
       ----- */

```

```

size_t new_size = 200;

char *new_buffer = mremap(
    buffer,
    size,
    new_size,
    MREMAP_MAYMOVE
);

if (new_buffer == MAP_FAILED)
{
    perror("mremap");
    munmap(buffer, size);
    return 1;
}

buffer = new_buffer;
size = new_size;

printf("\nMemory resized from 100 to 200 bytes\n");

/* -----
PART 3: Append Data
----- */

strncat(
    buffer,
    " - Memory Resized",
    size - strlen(buffer) - 1
);

printf("Updated string: %s\n", buffer);

/* -----
PART 4: Linked List
----- */

struct Node *head = NULL;
struct Node *temp;

for (int i = 1; i <= 5; i++)
{
    temp = malloc(sizeof(struct Node));

    if (temp == NULL)

```

```

{
    perror("malloc");

    struct Node *current = head;

    while (current != NULL)
    {
        struct Node *next = current->next;
        free(current);
        current = next;
    }

    munmap(buffer, size);

    return 1;
}

temp->data = i * 10;
temp->next = head;
head = temp;
}

printf("\nLinked List:\n");

temp = head;

while (temp != NULL)
{
    printf("%d -> ", temp->data);
    temp = temp->next;
}

printf("NULL\n");

/* -----
PART 5: Free Linked List
----- */

temp = head;

while (temp != NULL)
{
    struct Node *next = temp->next;

    free(temp);

    temp = next;
}

```

```

}

printf("Linked list memory released using free()\n");

/* -----
PART 6: Release mmap Memory
----- */

if (munmap(buffer, size) == 0)
{
    printf("Memory released successfully using munmap()\n");
}
else
{
    perror("munmap");
}

return 0;
}

```

This implementation follows the memory-management sequence shown in your Task 5 diagram: allocate memory, read input, resize it, append data, create/display a linked list, free the linked list, and release the mapped memory.

```

GNU nano 6.2 task5.c
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <uring.h>
#include <unistd.h>
#include <sys/mman.h>

struct Node
{
    int data;
    struct Node *next;
};

int main()
{
    size_t size = 100;
    char *buffer = mmap(NULL, size, PROT_READ | PROT_WRITE,

```

## Step 6: Save the program

Inside Nano press:

Ctrl + O



At the bottom you'll see:

File Name to Write: task5.c

Press:

Enter

Then exit Nano:

Ctrl + X



```
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/src$
```

## Step 7: Check that task5.c exists

Type:

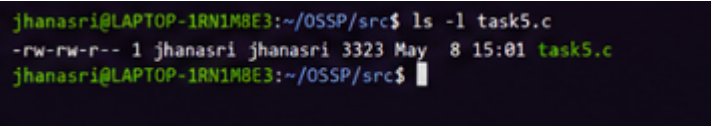
ls

You should now see:

task5.c

You can also use:

ls -l task5.c



```
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/src$ ls -l task5.c
-rw-rw-r-- 1 jhanasri jhanasri 3323 May  8 15:01 task5.c
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/src$
```

## Step 8: Compile the program

Type:

gcc task5.c -o task5

Press **Enter**.

**If there is no message:**

That's good.

It means compilation succeeded.

```
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/src$ gcc task5.c -o task5
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/src$
```

## Step 9: Check the executable

Type:

ls -l task5

You should see something similar to:

-rwxr-xr-x ... task5

```
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/src$ ls -l task5
-rwxrwxr-x 1 jhanasri jhanasri 22416 May  8 15:01 task5
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/src$
```

## Step 10: Run the program

Type:

./task5

You should see:

Allocated 100 bytes using mmap()

Enter some text:

```
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/src$ ./task5
Allocated 100 bytes using mmap()
Enter some text:
```

## Step 11: Enter some text

Type:

Hello Linux

Press **Enter**.

You should get something similar to:

Allocated 100 bytes using mmap()

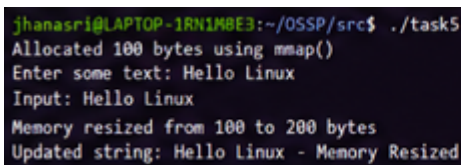
Enter some text: Hello Linux

Input: Hello Linux

Memory resized from 100 to 200 bytes

Updated string: Hello Linux

- Memory Resized

A terminal window screenshot showing the execution of a program. The prompt is 'jhanasri@LAPTOP-1RN1M8E3:~/OSSP/src\$ ./task5'. The output lines are: 'Allocated 100 bytes using mmap()', 'Enter some text: Hello Linux', 'Input: Hello Linux', 'Memory resized from 100 to 200 bytes', and 'Updated string: Hello Linux - Memory Resized'.

```
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/src$ ./task5
Allocated 100 bytes using mmap()
Enter some text: Hello Linux
Input: Hello Linux
Memory resized from 100 to 200 bytes
Updated string: Hello Linux - Memory Resized
```

## Step 12: Check the Linked List

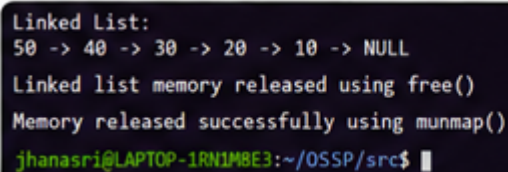
The program then creates 5 linked-list nodes.

You should see:

Linked List:

50 -> 40 -> 30 -> 20 -> 10 -> NULL

The exact order comes from inserting each new node at the beginning.

A terminal window screenshot showing the output of the linked list program. The output lines are: 'Linked List:', '50 -> 40 -> 30 -> 20 -> 10 -> NULL', 'Linked list memory released using free()', 'Memory released successfully using munmap()', and the prompt 'jhanasri@LAPTOP-1RN1M8E3:~/OSSP/src\$'.

```
Linked List:
50 -> 40 -> 30 -> 20 -> 10 -> NULL
Linked list memory released using free()
Memory released successfully using munmap()
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/src$
```

## Step 13: Check memory release

You should then see:

Linked list memory released using free()

Memory released successfully using munmap()

So your complete output will look approximately like:

Allocated 100 bytes using mmap()

Enter some text: Hello Linux

Input: Hello Linux

Memory resized from 100 to 200 bytes

Updated string: Hello Linux

- Memory Resized

Linked List:

50 -> 40 -> 30 -> 20 -> 10 -> NULL

Linked list memory released using free()

Memory released successfully using munmap()

```
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/src$ valgrind --leak-check=full ./task5
==15736== Memcheck, a memory error detector
==15736== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==15736== Using Valgrind-3.21.0 and LibVEX; rerun with -h for copyright info
==15736== Command: ./task5
Allocated 100 bytes using mmap()
Enter some text: Hello Linux
Input: Hello Linux
Memory resized from 100 to 200 bytes
Updated string: Hello Linux - Memory Resized
Linked List:
50 -> 40 -> 30 -> 20 -> 10 -> NULL
Linked list memory released using free()
Memory released successfully using munmap()
```

## Step 14: Install Valgrind if necessary

Now we need to validate the program as required by your Task 5 document.

First check whether Valgrind is installed:

valgrind --version

**If you get something like:**

valgrind-3.xx.x

Valgrind is already installed.

**If you get:**

valgrind: command not found

Install it:

sudo apt update

Then:

sudo apt install valgrind

If it asks:

Do you want to continue? [Y/n]

Type:

Y

Press **Enter**.

```
==15736==  
==15736== HEAP SUMMARY:  
==15736==    in use at exit: 0 bytes in 0 blocks  
==15736== total heap usage: 6 allocs, 6 frees, 1,344 bytes allocated  
==15736== All heap blocks were freed -- no leaks are possible  
==15736==  
==15736== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)  
  
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/src$
```

## Step 15: Run Valgrind

Now execute:

```
valgrind --leak-check=full ./task5
```

When it asks for input:

Enter some text:

Enter:

Hello Linux

## Step 16: Check the Valgrind result

At the bottom, you should see information similar to:

```
HEAP SUMMARY:  
    in use at exit: 0 bytes in 0 blocks  
    total heap usage: ...
```

All heap blocks were freed -- no leaks are possible

You may also see:

ERROR SUMMARY: 0 errors from 0 contexts

That's the result you want.

## Step 17: Final verification

Run:

ls -l

You should have:

```
task1
task1.c
task2
task2.c
task3
task3.c
task4
task4.c
task5
task5.c
```